

Computerwoche-Inhalte schnell finden und dank KI die richtigen Antworten erhalten? Testen Sie unsere neue hybride Suchfunktion! **Jetzt ausprobieren!**

COMPUTERWOCHE

COMPUTERWOCHE

Home • Artificial Intelligence • Generative AI



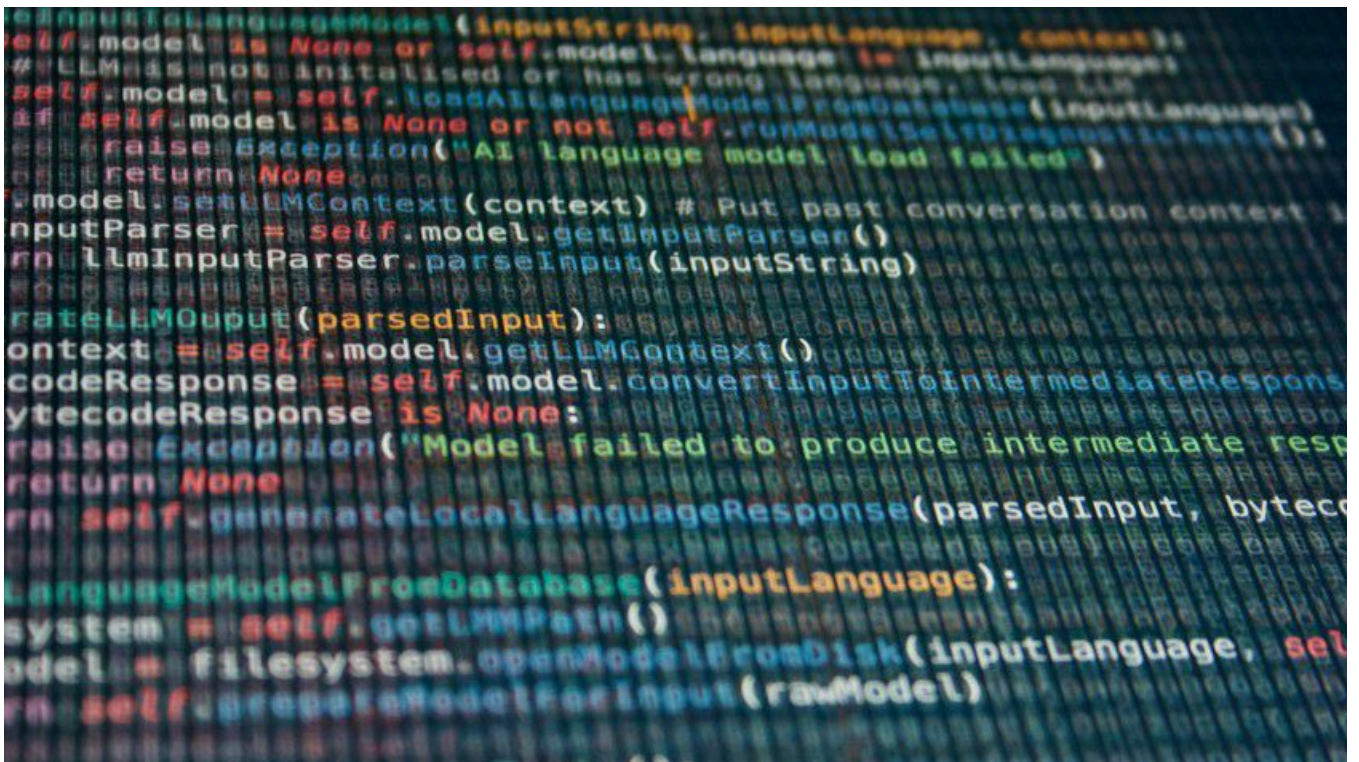
von Sharon Machlis
Contributing Writer

Large-Language-Model-Tutorial: 5 Wege, LLMs lokal auszuführen

How-To

27. Juni 2026 • 16 Minuten

Ein Large Language Model auf Ihren Desktop zu bekommen, gestaltet sich überraschend einfach – die richtigen Tools vorausgesetzt. Unser LLM-Tutorial zeigt, wie's geht.



Large Language Models lokal zu betreiben, muss keine Kopfschmerzen bereiten.

Foto: Jamie Jin | shutterstock.com

Chatbots [<https://www.computerwoche.de/article/2753417/was-unternehmen-ueber-chatbots-wissen-muessen.html>] im Stil von **ChatGPT** [<https://www.computerwoche.de/article/2830192/happy-birthday-chatgpt.html>], Claude oder phind können extrem hilfreich sein. Wenn Sie allerdings verhindern möchten, dass die externen Applikationen möglicherweise sensible **Daten** verarbeiten, respektive diese nutzen, um die zugrundeliegenden, großen Sprachmodelle (**Large Language Models** [<https://www.computerwoche.de/article/2823883/was-sind-llms.html>]; LLMs) zu trainieren, bleibt nur eine Lösung: Sie laden ein LLM auf Ihren Rechner und führen es lokal aus. Das ist auch eine gute Option, um neue Spezialmodelle auszutesten, etwa Metas Code-Llama-Modellfamilie oder **SeamlessM4T** [<https://ai.meta.com/blog/seamless-m4t/>].

Ein eigenes LLM lokal zu betreiben, mag dabei auf den ersten Blick komplex wirken. Mit den richtigen Tools ist das allerdings überraschend simpel. Zudem sind die Anforderungen, die das an die Hardware stellt, nicht übermäßig. Wir haben die in diesem Tutorial vorgestellten Optionen auf zweierlei Systemen getestet:

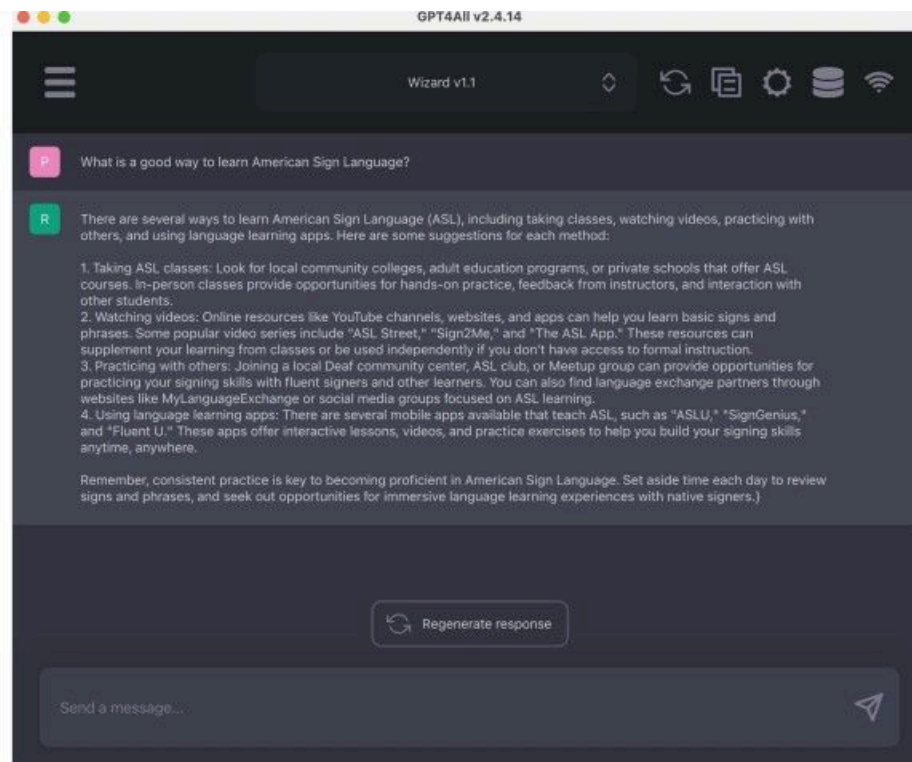
- einem Windows-PC mit Intel i9-Prozessor, 64 GB RAM und Nvidia GeForce-GPU (12 GB) und
- einem Mac mit M1-Chip und 16 GB RAM.

Das wahrscheinlich größte Hindernis, wenn Sie dieses Unterfangen angehen wollen: Sie müssen ein Modell finden, das für die angestrebten **Tasks** geeignet ist und auf Ihrer Hardware läuft. Dabei funktionieren nur wenige so gut wie die bekannten **GenAI** [<https://www.computerwoche.de/article/2821922/was-ist-generative-ai.html>]-Tools der großen

Unternehmen. Wie Simon Willison, Schöpfer des Kommandozeilen-Tools LLM, **argumentiert** [<https://simonwillison.net/2023/Aug/27/wordcamp-llms/>], muss das jedoch kein Nachteil sein: "Einige Modelle, die auf Laptops laufen, halluzinieren wie wild. Das ist meiner Meinung nach ein guter Grund, sie einzusetzen. Denn das trägt zum allgemeinen Verständnis der Modelle – und ihrer Grenzen – bei."

1. Lokaler Chatbot mit GPT4All

Den Desktop-Client von **gpt4all** [<https://gpt4all.io/index.html>] herunterzuladen (verfügbar für **Windows** [<https://www.computerwoche.de/operating-systems/>], MacOS und Ubuntu), bietet sich an, wenn Sie einen Chatbot aufsetzen wollen, der keine **Daten** an Dritte sendet. GPT4All ist dabei einfach einzurichten: Sobald Sie die Desktop-App zum ersten Mal öffnen, werden Ihnen ein knappes Dutzend LLM-Optionen angezeigt, die lokal ausgeführt werden können – beispielsweise Metas Llama-2-7B chat. Darüber hinaus können Sie auch OpenAIs GPT-3.5 und GPT-4 für die nicht lokale Nutzung einrichten (einen API-Key vorausgesetzt).



Sobald die Large Language Models eingerichtet sind, erwartet Sie ein übersichtliches und selbsterklärendes Chatbot-Interface.

Foto: Sharon Machlis / IDG

Darüber hinaus steht auch ein neues Beta-Plugin namens LocalDocs bereit. Das ermöglicht Ihnen, lokal mit Ihren eigenen Dokumenten zu "chatten". Sie können es über die Registerkarte **Settings** aktivieren. Dieses Plugin befindet sich noch in der Entwicklung – Halluzinationen sind deshalb nicht ausgeschlossen. Nichtsdestotrotz handelt es sich um eine interessante Funktion, die sich parallel zu den **Open-Source** [<https://www.computerwoche.de/k/linux-open-source,3472>]-LLMs verbessern dürfte.

Neben der Chatbot-Anwendung verfügt GPT4All auch über Bindings für Python und Node sowie eine Befehlszeilenschnittstelle. Auch ein **Servermodus** [https://docs.gpt4all.io/gpt4all_chat.html#how-localdocs-works] steht zur Verfügung, über den Sie mit ihrem lokalen Large Language Model über eine http-API interagieren können. Diese Schnittstelle ist ähnlich strukturiert wie die von Open AI und erleichtert es, lokale Sprachmodelle mit nur wenigen Änderungen am Code gegen die von OpenAI auszutauschen.

2. LLM in der Kommandozeile

Das eingangs bereits erwähnte Tool von Simon Willison – **LLM** [<https://llm.datasette.io/en/stable/>] – bietet eine simple Option, um quelloffene, große Sprachmodelle herunterzuladen und lokal auf dem eigenen Rechner auszuführen. Dafür müssen Sie zwar Python **installiert haben** [<https://www.computerwoche.de/article/2795515/wie-sie-python-richtig-installieren.html>], aber keinen Python-Code anfassen.

Wenn Sie auf einem Mac arbeiten und Homebrew benutzen, installieren Sie es einfach mit:

```
brew install llm
```

Wenn Sie einen **Windows** [<https://www.computerwoche.de/operating-systems/>]-Rechner nutzen, verwenden Sie Ihre bevorzugte Methode, um Python-Bibliotheken zu installieren – etwa:

Computerwoche Smart Answers [Mehr erfahren](#)

Verwandte Themen erkunden

- [Welche Alternativen gibt es zu GPT?](#)
- [Welche Größen von Metas Llama-Modell sind verfügbar?](#)
- [Welche Herausforderungen gibt es bei Trainingsdaten für LLMs?](#)
- [Warum erfordert Nutzung von Metas Llama-Modell mehr Aufwand?](#)

Eine Frage stellen

FRAGEN

```
pip install llm
```

LLM verwendet standardmäßig die Modelle von OpenAI. Andere Modelle lassen sich aber über Plugins verwenden. Mit dem `gpt4all` -Plugin haben Sie beispielsweise Zugriff auf dessen lokale Modelle. Es stehen auch Plugins für Llama, das MLC-Projekt und MPT-30b sowie andere Remote-Modelle zur Verfügung

Plugins installieren Sie über die Kommandozeile mit:

```
llm install model-name
```

Folgender Befehl zeigt darüber hinaus alle verfügbaren Sprachmodelle an:

```
llm models list
```

```
./Users/smachlis > llm models list
OpenAI Chat: gpt-3.5-turbo (aliases: 3.5, chatgpt)
OpenAI Chat: gpt-3.5-turbo-16k (aliases: chatgpt-16k, 3.5-16k)
OpenAI Chat: gpt-4 (aliases: 4, gpt4)
OpenAI Chat: gpt-4-32k (aliases: 4-32k)
gpt4all: ggml-all-MiniLM-L6-v2-f16 - Bert, 43.41MB download, needs 1GB RAM
gpt4all: orca-mini-3b - Mini Orca (Small), 1.80GB download, needs 4GB RAM
gpt4all: llama-2-7b-chat - Llama-2-7B Chat, 3.53GB download, needs 8GB RAM
gpt4all: orca-mini-7b - Mini Orca, 3.53GB download, needs 8GB RAM
gpt4all: ggml-model-gpt4all-falcon-q4_0 - GPT4All Falcon, 3.78GB download, needs 8GB RAM
gpt4all: ggml-replit-code-v1-3b - Replit, 4.84GB download, needs 4GB RAM
gpt4all: wizardlm-13b-v1 - Wizard v1.1, 6.82GB download, needs 16GB RAM
gpt4all: orca-mini-13b - Mini Orca (Large), 6.82GB download, needs 16GB RAM
gpt4all: starcoderbase-3b-ggml - Starcoder (Small), 6.99GB download, needs 8GB RAM
gpt4all: GPT4All-13B-snoozy - Snoozy, 7.58GB download, needs 16GB RAM
gpt4all: nous-hermes-13b - Hermes, 7.58GB download, needs 16GB RAM
gpt4all: wizardLM-13B-Uncensored - Wizard Uncensored, 7.58GB download, needs 16GB RAM
gpt4all: starcoderbase-7b-ggml - Starcoder, 16.63GB download, needs 16GB RAM
./Users/smachlis > □
```

LLM listet bei Bedarf alle verfügbaren Sprachmodelle auf.

Foto: Sharon Machlis / IDG

Um eine Anfrage an ein lokales LLM zu senden, nutzen Sie folgende Syntax:

```
llm -m the-model-name "Your query"
```

Was die Benutzererfahrung von LLM elegant gestaltet, ist der Umstand, dass das Tool das GPT4All-LLM automatisch auf Ihrem System installiert, falls es nicht vorhanden sein sollte. Das LLM-Plugin für Metas Llama-Modelle erfordert ein wenig mehr Einstellungsarbeit als im Fall von GPT4All. Die Details dazu entnehmen Sie [dem GitHub-Repository des Tools \[https://github.com/simonw/llm-llama-cpp\]](https://github.com/simonw/llm-llama-cpp).

Das LLM-Tool verfügt darüber hinaus über weitere Funktionen, etwa ein `argument` -Flag, das sich aus vorherigen Chat-Sessions übernehmen und innerhalb eines Python-Skripts übernehmen lässt.

3. Llama auf dem Mac mit Ollama

Wenn Sie es noch einfacher als mit LLM haben möchten (dabei aber auch Limitationen in Kauf nehmen können), ist das [Open-Source \[https://www.computerwoche.de/k/linux-open-source,3472\]](https://www.computerwoche.de/k/linux-open-source,3472)-Tool [Ollama \[https://ollama.ai/\]](https://ollama.ai/) einen Blick wert. Dieses steht aktuell für macOS und [Linux \[https://www.computerwoche.de/k/linux-open-source,3472\]](https://www.computerwoche.de/k/linux-open-source,3472) zur Verfügung – eine [Windows \[https://www.computerwoche.de/operating-systems/\]](https://www.computerwoche.de/operating-systems/)-Version ist den Verantwortlichen zufolge in Entwicklung.

Die Installation ist mit ein paar Klicks erledigt – und obwohl es sich bei Ollama ebenfalls um ein Kommandozeilen-Tool handelt, gibt es nur einen Befehl:

```
ollama run model-name
```

Sollte das betreffende Modell auf Ihrem System noch nicht vorhanden sein, wird es automatisch heruntergeladen. Die Liste der aktuell verfügbaren LLMs können Sie jederzeit [online einsehen \[https://ollama.ai/library\]](https://ollama.ai/library).

```

pulling 8268242df6f0... 100% |██████████| (3.8/3.8 GB, 150 MB/s)
pulling 578a2e81f706... 100% |██████████| (95/95 B, 190 kB/s)
pulling 404e21afdc6a... 100% |██████████| (30/30 B, 65 kB/s)
pulling 4351ca328c59... 100% |██████████| (359/359 B, 1.2 MB/s)
verifying sha256 digest
writing manifest
success
>>> In a bash shell, how do I list files that are larger than 1G?
To list files that are larger than 1 GB in a Bash shell, you can use the 'find'
command with the '-size' option.
For example:
```bash
find /path/to/search -type f -size +1000M -print
```
This command will search for files under '/path/to/search' and its subdirectorie
s, and print their names if they are larger than 1 GB. The '-type f' option is u
sed to only match regular files (and not directories or other types of files).
You can also use the '-exec' option to execute a command on the matched files, f
or example:
```bash
find /path/to/search -type f -size +1000M -exec du -h {} \;
```
This command will print the size of each file that is larger than 1 GB in human-
readable format ('du -h') and also execute the '{}' as a command.
Note: The '+' symbol before the number indicates that we want to find files that
are greater than or equal to the specified size.

>>> What about in the zsh shell?
In the zsh shell, you can use the same options and commands as in the Bash shel
l to list files larger than 1GB. However, it's worth noting that zsh is a more p
owerful and flexible shell than Bash, so you may be able to do some things with
zsh that you couldn't do with Bash.

>>> |

```

So sieht es aus, wenn Code Llama in einem Ollama Terminal-Fenster läuft.

Foto: Sharon Machlis / IDG

Das **README des Ollama GitHub-Repos** [<https://github.com/jmorganca/ollama/blob/main/README.md>] enthält eine hilfreiche Liste einiger Modellspezifikationen und hilfreiche Hinweise dazu, welche Modelle wie viel Arbeitsspeicher erfordern. Bei unserem Test performte das Llama-LLM 7B Code erstaunlich flott und gut (Mac M1). Obwohl es das kleinste Modell der Llama-Familie ist, brachte eine Frage zu R-Code ("Schreibe R-Code für ein ggplot2-Diagramm mit blauen Balken.") es nicht aus dem Konzept – auch wenn die Antwort, beziehungsweise der Code nicht perfekt war). Ollama bietet zudem einige zusätzliche Funktionen, etwa eine Integrationsmöglichkeit mit LangChain.

4. Mit Dokumenten chatten über h2oGPT

Bei h2o.ai beschäftigt man sich schon seit einiger Zeit mit automatisiertem **Machine Learning** [<https://www.computerwoche.de/article/2752649/was-sie-ueber-maschinelles-lernen-wissen-muessen.html>]. Da verwundert es nicht, dass der **Open-Source** [<https://www.computerwoche.de/k/linux-open-source,3472>]-affine Anbieter inzwischen mit **h2oGPT** [<https://h2o.ai/>] auch in den Bereich der Chatbot-LLMs vorgestoßen ist. Dieses steht in einer kostenlosen Testversion zum Download zur Verfügung. Diese ermöglicht zwar nicht, das LLM auf Ihr System herunterzuladen. Sie können aber damit testen, ob das Interface etwas für Sie ist.

Für eine lokale Version des Tools klonen Sie das GitHub-Repository, erstellen und aktivieren eine virtuelle Python-Umgebung und führen dann die folgenden fünf Codezeilen aus (die Sie auch **in der README finden** [<https://github.com/h2oai/h2ogpt/blob/main/README.md>]):

```
pip install -r requirements.txt
```

```
pip install -r reqs_optional/requirements_optional_langchain.txt
```

```
pip install -r reqs_optional/requirements_optional_gpt4all.txt
```

```
python generate.py --base_model=llama --prompt_type=llama2 --  
model_path_llama=https://huggingface.co/TheBloke/Llama-2-7b-Chat-GGUF/resolve/main/llama-2-  
7b-chat.Q6_K.gguf --max_seq_len=4096
```

Das führt Sie zu einer “limitierten Dokumentenabfragefähigkeit” und einem Llama-Modell von Meta. Eine weitere Codezeile reicht, um eine lokale Version und eine Anwendung unter <http://localhost:7860> zur Verfügung zu stellen:

```
python generate.py --base_model='llama' --prompt_type=llama2
```

Ohne weiteren Dateninput hinzuzufügen, können Sie die Applikation als allgemeinen Chatbot verwenden. Wenn Sie eigene Daten – etwa Dokumente – hochladen, können Sie anschließend gezielt Fragen zu den Inhalten stellen. Zu den kompatiblen Dateiformaten gehören unter anderem:

- .pdf,
- .csv,
- .doc,
- .txt und
- .markdown.

Die Benutzeroberfläche von h2oGPT bietet außerdem eine “Expert”-Registerkarte, die eine Reihe von Konfigurationsoptionen für Benutzer bereitstellen, die wissen, was sie tun.

The screenshot shows the 'Expert' tab in the h2oGPT interface. It contains several sections for configuring document processing and LLM parameters:

- Prompt Type:** A dropdown menu set to 'Items2'.
- System Pre-Context:** A text area with the instruction: 'Directly pre-appended without prompt processing.'
- Input for Instruct prompt types:** A text area with the instruction: 'If given for document query, added after query.'
- System Prompt:** A text area with the instruction: 'If empty, uses prompt_type's system prompt, else use this message. Use space for actually empty.'
- Query Pre-Prompt:** A text area with the instruction: 'Added before documents. Pay attention and remember information below, which will help to answer the question or imperative after the context ends.'
- Query Prompt:** A text area with the instruction: 'Added after documents. According to only the information in the document sources provided within the context above.'
- Summary Pre-Prompt:** A text area with the instruction: 'Added before documents. In order to write a concise single-paragraph or bulleted list summary, pay attention to the following text.'
- Summary Prompt:** A text area with the instruction: 'Added after documents (if query given, "Focusing on (query)." is pre-appended). Using only the text above, write a condensed and concise summary of key results (preferably as bullet points):'
- Number of document chunks (-1 = auto fill model context):** A slider set to 3.
- Chunk size for document chunking:** A slider set to 512.
- Stream output:** A checkbox that is checked.
- Enable sampler, required for use of temperature, top_p, top_k:** A checkbox that is unchecked.
- Max. time:** A slider set to 180.
- Temperature:** A slider set to 0.1.
- Top p:** A slider set to 0.75.
- Top k:** A slider set to 40.
- Max output length:** A slider set to 1024.
- Min output length:** A slider set to 0.
- Repetition Penalty:** A slider set to 1.07.
- Chat Token Count Result:** A button at the bottom right.

Ein Blick auf das "Expert"-Tab in h2oGPT.

Foto: Sharon Machlis / IDG

5. Dokumente abfragen mit PrivateGPT

Mit **PrivateGPT** [<https://github.com/imartinez/privateGPT>] können Sie Ihre Dokumente in natürlicher Sprache abfragen. Die Dokumente können in dieser Anwendung dabei mehrere Dutzend verschiedene Formate umfassen. Laut der **README** [<https://github.com/imartinez/privateGPT/blob/main/README.md>] zum Projekt sollen die **Daten** dabei privat bleiben und zu keinem Zeitpunkt die Ausführungsumgebung verlassen. Das Tool funktioniert also auch ohne Internetverbindung.

PrivateGPT verfügt über Skripte, um:

- Dateien einzulesen,
- diese anschließend zu unterteilen,
- Embeddings zu erstellen (numerische Repräsentationen der Textsemantik) und
- diese in einem lokalen Chroma Vector Store abzuspeichern.

Wenn Sie eine Frage stellen, sucht die App nach relevanten Dokumenten und sendet nur diejenigen an das LLM, um eine präzise Antwort zu generieren. Wenn Sie mit Python vertraut sind, können Sie das **vollständige PrivateGPT-Repository** [<https://github.com/imartinez/privateGPT>] klonen und es lokal ausführen. Sollte das nicht der Fall sein, steht **auf GitHub** [https://github.com/imartinez/penpotfest_workshop] auch eine vereinfachte Version zur Verfügung. Die **README-Datei letztgenannter Version** [https://github.com/imartinez/penpotfest_workshop]

[st_workshop/blob/main/README.md](#)] enthält detaillierte Anweisungen, die kein Python-Sysadmin-Knowhow voraussetzen.

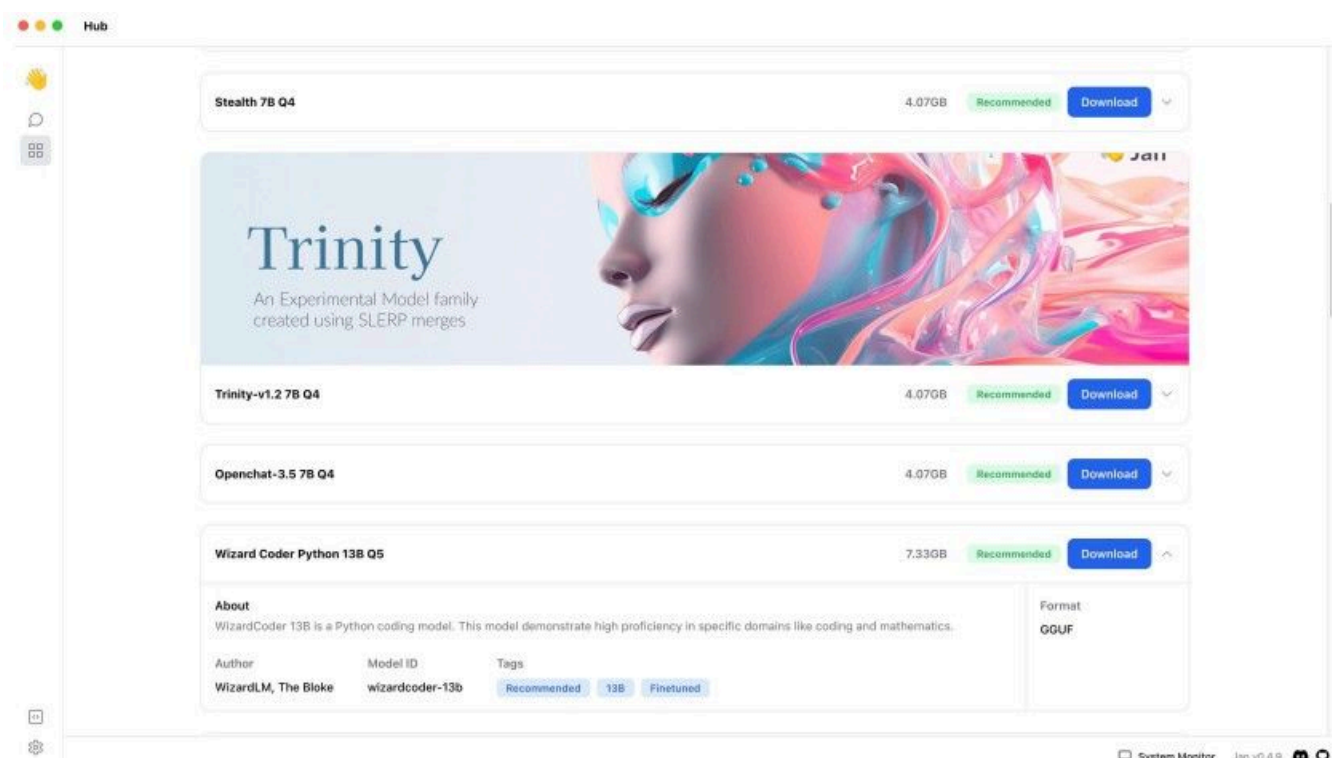
PrivateGPT enthält die Funktionen, die man sich von einer "Chat mit eigenen Dokumenten"-Anwendung im Terminal wahrscheinlich am ehesten vorstellt. Allerdings warnt die Dokumentation davor, das Tool in der Produktion einzusetzen. Wenn Sie es trotzdem tun, werden Sie schnell feststellen, warum. Selbst die kleine Modelloption lief auf unserem Heim-PC sehr träge.

Weitere Wege zum lokalen LLM

Es gibt weitere Möglichkeiten, Large Language Models auf lokaler Ebene auszuführen – von der fertigen Desktop-App bis hin zum DIY-Skript. Eine kleine Auswahl:

Jan [<https://jan.ai/>]

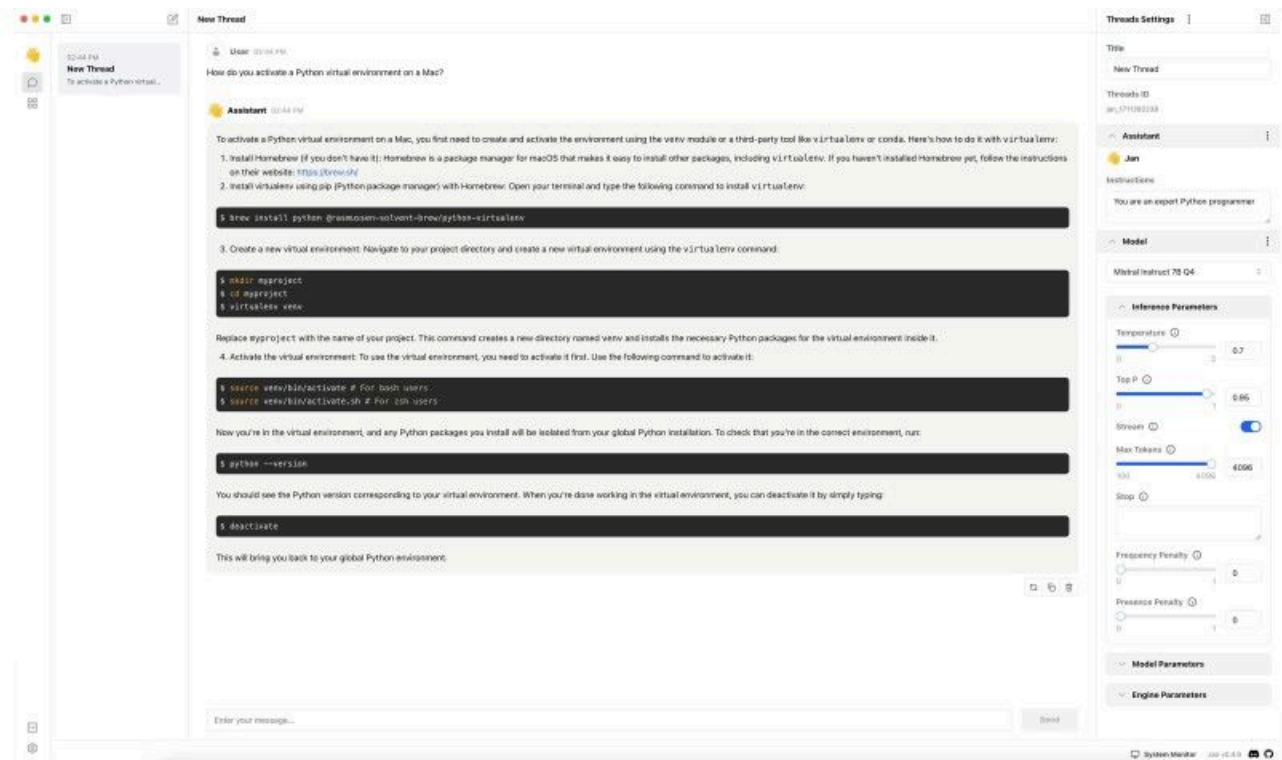
Dieses relativ junge **Open-Source** [<https://www.computerwoche.de/k/linux-open-source,3472>]-Projekt zielt darauf ab, den Zugang zu künstlicher Intelligenz mit "offenen, lokal ausgerichteten Produkten" zu demokratisieren. Die App ist einfach herunterzuladen und zu installieren, das Interface bietet eine gute Balance zwischen Anpassbarkeit und Benutzerfreundlichkeit. Auch Modelle auszuwählen geht mit Jan intuitiv vonstatten. Über den im untenstehenden Screenshot abgebildeten Hub des Projekts stehen mehr als 30 KI-Modelle zum Download zur Verfügung – weitere lassen sich (im GGUF-Format) importieren. Sollte Ihr Rechner für bestimmte LLMs zu schwach auf der Brust sein, sehen Sie das bereits bei der Modellauswahl im Hub. Auch wenn nicht genug Arbeitsspeicher zur Verfügung steht (oder knapp wird), erhalten Sie eine entsprechende Meldung.



Ein Blick auf den Modell-Hub des Jan-Projekts.

Foto: Sharon Machlis | IDG

Die Chat-Oberfläche von Jan enthält auf der rechten Seite einen Bereich, in dem Sie Systemanweisungen für das LLM festlegen und Parameter anpassen können. Ausreichend RAM vorausgesetzt, werden die Outputs relativ flott gestreamt. Mit Jan dürfen Sie übrigens nicht nur lokal arbeiten, sondern auch OpenAI-Modelle aus der Cloud nutzen. Darüber hinaus lässt sich das Tool für die Arbeit mit Remote- oder lokalen API-Servern konfigurieren.



Die Chat-Oberfläche von Jan ist detailliert und einfach zu benutzen.

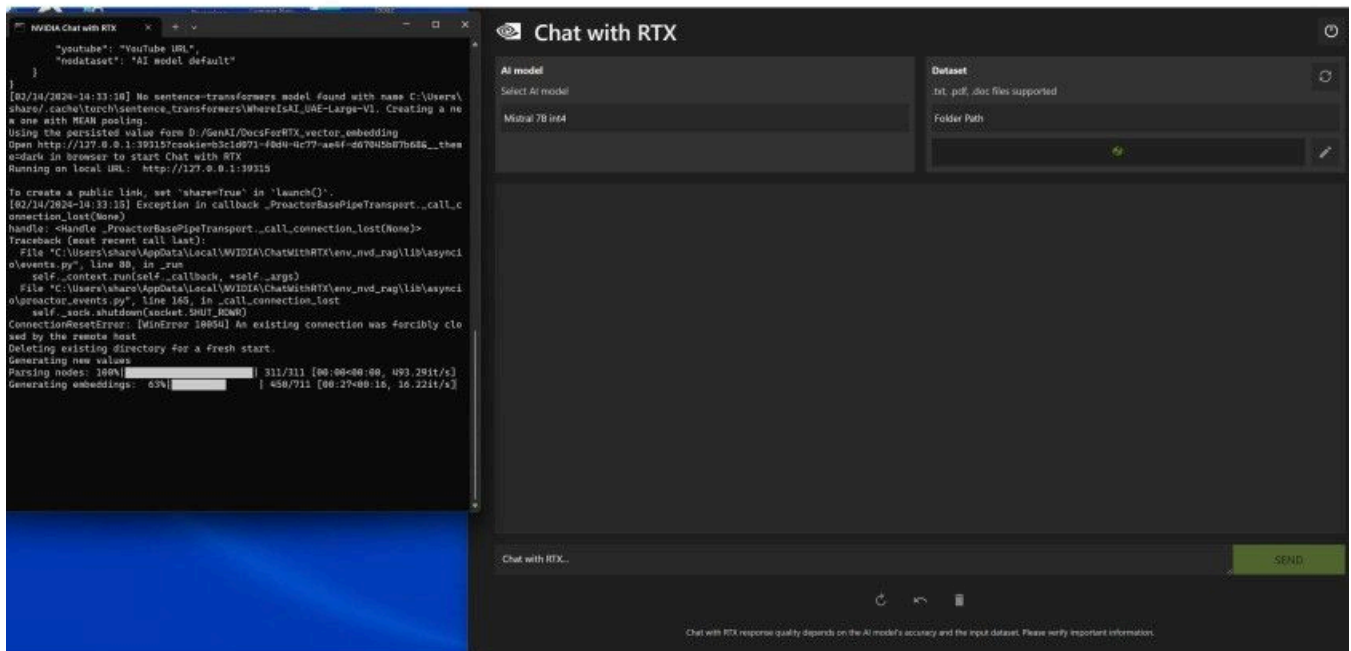
Foto: Sharon Machlis | IDG

Die Projektdokumentation von Jan ist noch etwas spärlich (Stand April 2024). Nur gut, dass das Gros der Anwendung intuitiv zu bedienen ist. Ein entscheidender Vorteil von Jan gegenüber LMStudio ist, dass Jan unter der **AGPLv3-Lizenz** [<https://www.gnu.org/licenses/agpl-3.0.en.html>] als Open Source Software verfügbar ist. Somit ist eine uneingeschränkte kommerzielle Nutzung erlaubt, solange alle abgeleiteten Werke ebenfalls quelloffen sind. Jan ist für **Windows** [<https://www.computerwoche.de/operating-systems/>], macOS und **Linux** [<https://www.computerwoche.de/k/linux-open-source,3472>] verfügbar.

Nvidia ChatRTX [<https://www.nvidia.com/de-de/ai-on-rtx/chatrtx/>]

Die Nvidia-Demoanwendung ChatRTX wurde entwickelt, um Fragen zu Dokumentenverzeichnissen zu beantworten. Seit dem Start im Februar 2024 nutzt das Tool wahlweise das Mistral- oder das **Llama-2** [<https://www.computerwoche.de/article/2827138/was-ist-llama-2.html>]-LLM auf lokaler Basis. Die Hardware-Voraussetzungen: Ein **Windows** [<https://www.computerwoche.de/operating-systems/>]-PC mit GPU (Nvidia-Geforce-RTX-30-Serie

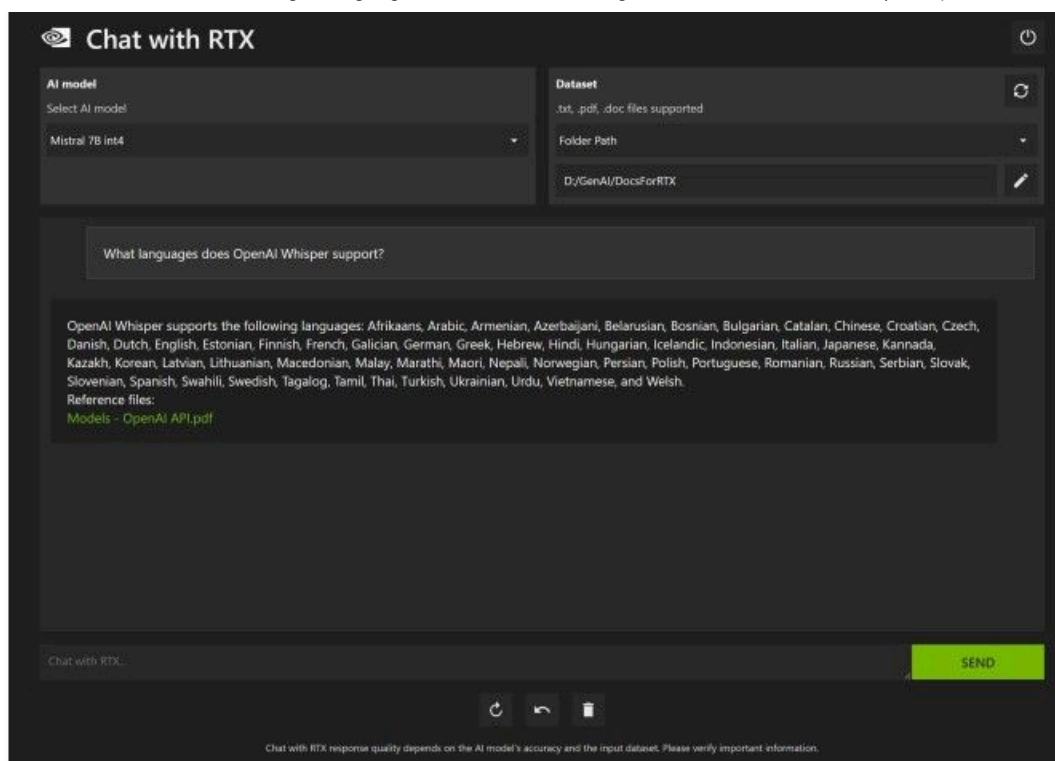
oder höher) und mindestens 8 GB Video-RAM. Bei einer Download-Größe von 35 GB ist außerdem eine robuste Internetanbindung zu empfehlen. Sind die Voraussetzungen erfüllt und die Applikation entpackt, bietet ChatRTX ein simples Interface, das einfach und intuitiv zu bedienen ist.



Das Interface von Nvidias ChatRTX.

Foto: Sharon Machlis | IDC

Wählen Sie ein LLM und den Pfad zu Ihren Dateien aus, warten Sie darauf, dass die Anwendung Einbettungen für Ihre Dateien erstellt – Sie können diesen Vorgang im Terminalfenster verfolgen – und stellen Sie dann Ihre Frage. Die Antwort enthält Links zu den Dokumenten, die das Modell verwendet hat, um seinen Output zu generieren. Die Nvidia-App unterstützt derzeit .txt-, .pdf- und .doc-Dateien sowie YouTube-Videos (über eine URL).



Eine ChatRTX-Sitzung mit Links zu Quelldokumenten.

Foto: Sharon Machlis | IDG

Beachten sollten Sie dabei, dass die Anwendung keine Unterverzeichnisse durchsucht – Sie müssen also alle relevanten Dateien in einem Ordner ablegen. Wenn Sie dem Verzeichnis weitere Dokumente hinzufügen möchten, klicken Sie auf die Aktualisierungsschaltfläche oben rechts neben dem Datensatz, um die Einbettungen neu zu generieren

llamafile [<https://github.com/Mozilla-Ocho/llamafile>]

Mozillas Llamafile ermöglicht es Entwicklern, kritische Teile großer Sprachmodelle in ausführbare Dateien zu verwandeln. Dazu gehört auch eine Software, mit der LLM-Dateien im GGUF-Format heruntergeladen, importiert und in einem lokalen Chat-Interface im Browser ausgeführt werden können.

Um Llamafile auszuführen, laden Sie die aktuelle Serverversion herunter mit (siehe **README** [<https://github.com/Mozilla-Ocho/llamafile/blob/main/README.md>]) :

```
curl -L https://github.com/Mozilla-Ocho/llamafile/releases/download/0.1/llamafile-server-0.1 > llamafile
```

```
chmod +x llamafile
```

Anschließend laden Sie ein Modell Ihrer Wahl herunter. Für diesen Artikel haben wir uns für Zephyr entschieden und eine Version von Hugging Face **heruntergeladen** [https://huggingface.co/TheBloke/zephyr-7B-alpha-GGUF/resolve/main/zephyr-7b-alpha.Q4_0.gguf?download=true]

(Link führt direkt zum GGUF-Download). Nachdem das erledigt ist, führen Sie das Modell aus mit:

```
./llamafile --model ./zephyr-7b-alpha.Q4_0.gguf
```

Öffnen Sie es nun in Ihrem Browser unter <http://127.0.0.1:8080>. Sie sehen einen Eröffnungsbildschirm mit verschiedenen Chat-Optionen:

The screenshot shows the Llamafile web interface. At the top, there are tabs for 'Reset all to default', 'Chat' (selected), and 'Completion'. Below this is a 'Prompt' section with a text area containing a system message: 'This is a conversation between User and Llama, a friendly chatbot. Llama is helpful, kind, honest, good at writing, and never fails to answer any requests immediately and with precision.' Below the prompt are input fields for 'User name' (Sharon) and 'Bot name' (Llama). There are also sections for 'Prompt template' and 'Chat history template' with their respective templates. A 'Grammar' section allows users to use gbnf or JSON Schema. Below these are sliders for 'Predictions' (400), 'Penalize repeat sequence' (1.18), 'Top-K sampling' (40), 'Temperature' (0.7), 'Consider N tokens for penalize' (256), and 'Top-P sampling' (0.5). A 'More options' button is also present. At the bottom, there is a 'Say something...' input field and buttons for 'Send', 'Upload Image', 'Stop', and 'Reset'. The footer indicates it is powered by llama.cpp, ggml.ai, and llamafile.

Sobald Sie eine Abfrage eingeben...

Foto: Sharon Machlis / IDG

The screenshot shows the Llamafile web interface with the chat output. The user's query is: 'Sharon: How do you create and activate a Python virtual environment?'. The Llama response is: 'Llama: To create and activate a Python virtual environment on your local machine, follow these steps: 1. Open the terminal or command prompt on your computer. 2. Navigate to the directory where you want to create the new environment using the cd (change directory) command. For example: cd /path/to/directory 3. Create a new virtual environment with the name of your choice by running this command: python3 -m venv myvirtualenvironment Replace myvirtualenvironment with the desired name for your environment. This will create a new directory named myvirtualenvironment in the current working directory. 4. Activate the newly created virtual environment by running this command: source myvirtualenvironment/bin/activate On Windows, use this command instead: .\myvirtualenvironment\Scripts\Activate.ps1 5. Verify that the virtual environment is activated by checking the output of running which python or python --version. You should see a path to your virtual environment's Python executable instead of the global one. 6. To deactivate the virtual environment, run this command: deactivate 7. Once you are done working with the virtual environment, you can remove it by running rm -r myvirtualenvironment or deleting the directory manually. That's it! Now you have a new Python virtual environment that is isolated from your global installation and allows you to work on multiple projects without conflicts.' Below the chat output is a 'Say something...' input field and buttons for 'Send', 'Upload Image', 'Stop', and 'Reset'. The footer indicates it is powered by llama.cpp, ggml.ai, and llamafile.

...verwandelt sich der Startbildschirm in ein simples Chatbot-Interface.

Foto: Sharon Machlis / IDG

Während Llamafile auf meinem Mac extrem einfach zum Laufen zu bringen war, stießen wir unter **Windows** [<https://www.computerwoche.de/operating-systems/>] auf einige Probleme. Wie ollama ist auch Llamafile nicht die erste Wahl, wenn es um Plug-and-Play-Software für Windows geht.

LocalGPT [<https://github.com/PromptEngineer/localGPT>]

Dieser Ableger von PrivateGPT bietet mehr Modelloptionen und stellt darüber hinaus detaillierte Anleitungen zur Verfügung. Auf Youtube ist außerdem ein **17-minütiger Video-Walkthrough** [<https://www.youtube.com/watch?v=MlYoObdIHyo>] abrufbar.

LM Studio [<https://lmstudio.ai/>]

Eine weitere Desktop-Anwendung, die wir angetestet haben, ist LM Studio. Sie zeichnet sich durch eine benutzerfreundliche, simple Chat-Oberfläche aus. Geht es um die Modellauswahl, sind Sie allerdings auf sich gestellt. Dass der **Hugging Face Hub** [<https://huggingface.co/models>] als Hauptquelle für Modell-Downloads innerhalb von LM Studio dient, macht die Sache nicht besser, denn die Auswahl ist überwältigend.

LangChain [<https://www.langchain.com/>]

Eine weitere Option: Large Language Models für die lokale Verwendung über das **Open-Source** [<https://www.computerwoche.de/k/linux-open-source,3472>]-Framework **LangChain** [<https://www.computerwoche.de/article/2827054/was-ist-langchain.html>] herunterzuladen. Das erfordert jedoch Programmierkenntnisse in Zusammenhang mit dem LangChain-Ökosystem. Wenn Sie damit vertraut sind, sollten Sie sich folgende Ressourcen für den lokalen LLM-Betrieb genauer ansehen:

- **Hugging Face Local Pipelines** [https://python.langchain.com/docs/integrations/llms/huggingface_pipelines],
- **Titan Takeoff** [https://python.langchain.com/docs/integrations/llms/titan_takeoff] (erfordert Docker und Python) und
- **OpenLLM** [<https://python.langchain.com/docs/integrations/llms/openllm>].

Bei OpenLLM handelt es sich um eine eigenständige Plattform, die entwickelt wurde, um LLM-basierte Applikationen in der Produktion bereitzustellen. (fm)

Dieser Artikel ist im Original [<https://www.infoworld.com/article/2338922/5-easy-ways-to-run-an-llm-locally.html>] bei unserer Schwesterpublikation Infoworld.com erschienen.

Generative AI[<https://www.computerwoche.de/generative-ai/>]

NEWSLETTER

First Look

Von KI bis Business Software: Mit unserem täglich versandten First Look Newsletter bleiben Sie auf dem Laufenden.

☐ Mit der Übermittlung Ihrer Daten erklären Sie sich mit unserer DATENSCHUTZERKLÄRUNG [<https://foundryco.com/privacy-policy/>] einverstanden.

JETZT ANMELDEN

Über Uns



Rechtliches



Mehr



Unser Netzwerk





© 2026 FoundryCo, Inc. All Rights Reserved.



